SeBS-Flow:

Benchmarking Serverless Cloud Function Workflows

By Larissa Schmid, Marcin Copik, Alexandru Calotoiu,
Laurin Brandner, Anne Koziolek, Torsten Hoefler

From Karlsruhe Institute of Technology & ETH Zurich

Anna Makridou CS-559

Presentation Outline

01 — The Problem

What are they trying to solve?

02 — The Solution

What did the authors build?

03 — The Experiment

How did they test it?

04 — The Results

What did they find?

05 — The Conclusion

Why does it matter?

01

The Problem

What are they trying to solve?

What is Serverless Computing?

Developers write small, stateless functions and are no longer concerned with managing their applications.

- **Limitation**: A single function is insufficient for larger applications.
- **Solution**: Real applications must chain all their multiple functions together in **Serverless Workflows.**

The Core Problem: Incompatible Platforms

Serverless workflows are quite distinct from each other .

- They offer drastically different performance, reliability, and cost.

- This makes fair and consistent performance evaluations difficult.

Vendor lock-in

What is Vendor Lock-in?

- Being dependent on a single vendor, making it difficult and costly to switch to a competitor.

- The paper notes: "Moving workflows from one platform to another is complicated, causing vendor lock-in."

SeBS-Flow: Benchmarking Serverless Cloud Function Workflows

```python
tasks = []
for i in range(4):
  tasks.append(context.call_activity("process", i)
res = yield context.task_all(parallel_tasks)
```

(a) Azure Durable Functions

```
"assign_array": {
    "assign": [
      {"array": [0, 1, 2, 3]}]
},
"process": {
    "call":"exp.exec.map",
    "args":{
      "workflow_id":"map",
      "arguments":"${array}"
    },
    "result":"res"
}
"separate map-workflow:"
"main": {
"params":[ "elem" ],
"steps":[
  {
  "map":{
    "call":"http.post",
    "args":{
      "url":"google.process",
      "body":{
        "payload":"${elem}"
      }
    },
    "result":"elem" }
  },
  { "ret": {
      "return":
      "${elem.body}" } }
] }
```

(b) Google Cloud Workflows

```
"init": {
  "Type": "Pass",
  "Result": "States.Array(0,
            1, 2, 3)",
  "ResultPath": "$.array",
  "Next": "map"
},
"map": {
  "Type": "Map",
  "ItemsPath": "$.array",
  "Parameters": {
    "payload.$":
      "$$.Map.Item.Value"
  },
  "Iterator": {
    "StartAt": "process",
    "States": {
      "process": {
        "Type": "Task",
        "Resource": "arn:proc",
        "Parameters": {
          "payload.$":
            "$.payload"
        },
        "End": true
      }
    }
  },
  "ResultPath": "$.res",
  "End": true
}
```

(c) AWS Step Functions

02

The Solution

What did the authors build?

The Solution: The SeBS-Flow Toolkit

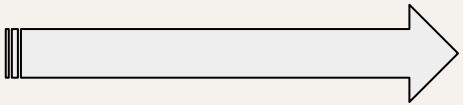
The first serverless workflows benchmarking toolkit.

Goal: Provide a baseline and benchmarking methodology for evaluating and comparing the performance of workflows.

Core Idea: A unified and portable workflow model that is platform-agnostic.

How SeBS-Flow Works: A Universal Translator

01



02

Define

Define workflow once using a simple JSON.

Translate

Convert the generic JSON file into the specific format of the cloud provider

What is the Generic JSON Model?

A simple JSON syntax that defines a workflow as a series of **phases**.

A **next** field in each phase defines the order, linking them together.

- Task: Runs a single function.
- Map: Runs functions in parallel over an array.
- Switch: Conditional logic (if/else).
- Loop: Runs functions in sequence.

03

The Experiment

How did they test it?

The Experiment: Realistic Workloads

Validation:

Analyzed **72** other research papers to ensure their model was flexible .

Proved their translator added **no noticeable overhead.** (13.6ms on a 3757s workflow) .

Benchmarks:

Used **6 real-world applications** and 4 microbenchmarks.

Video Analysis, Trip Booking (web app), MapReduce, Machine Learning (ML), 1000Genomes (science).

04

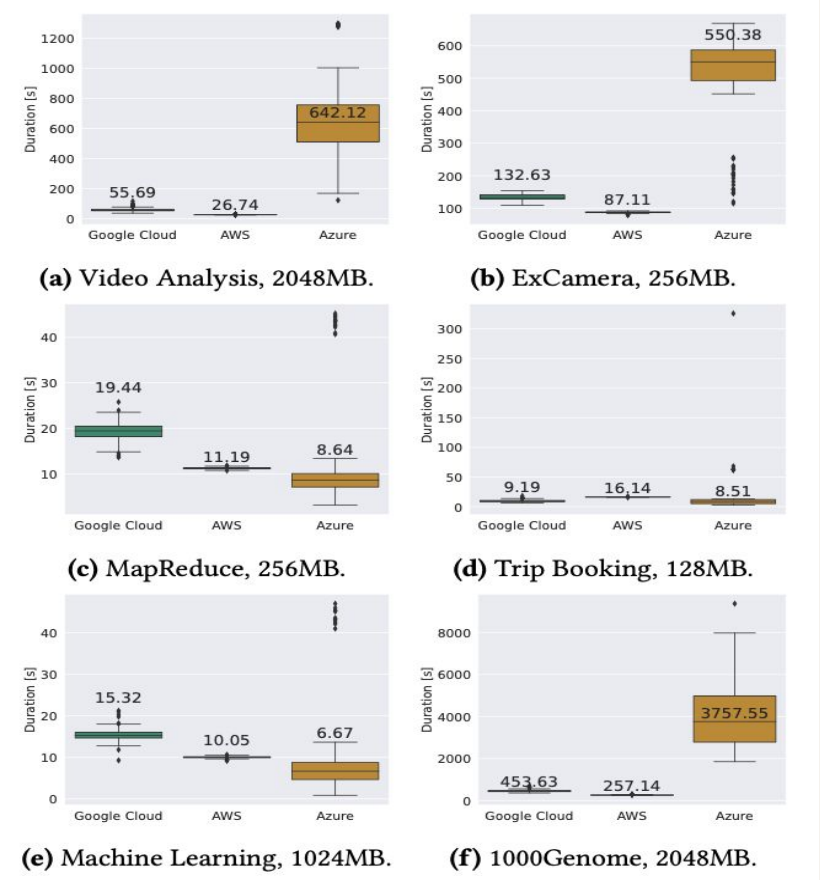
The Results

What did they find?

Who is fastest?

Finding:
There is **no single fastest platform**.

It completely depends on the workload!



Platform Breakdown

AWS

The best all-rounder. Fastest on **3 of 6** benchmarks.

AZURE

A gamble. Fastest for **2 of 6** benchmarks but disastrously slow 2 others.

GOOGLE CLOUD

Performance was **comparable to AWS**, but was 1.55-1.97x slower on three benchmarks.

Why is the performance so different?

The authors split runtime into two parts:

01

Critical Path

The actual compute time of the functions

02

Overhead

The orchestration time



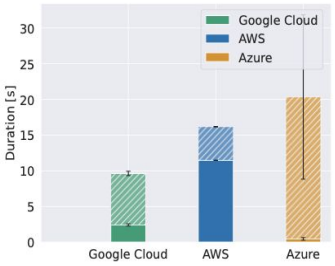
(a) Video Analysis, 2048MB.



(b) ExCamera, 256MB.

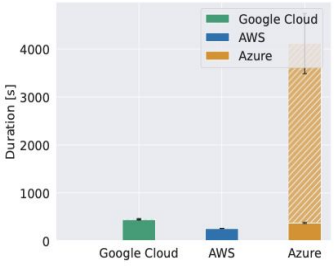


(c) MapReduce, 256MB.



(d) Trip Booking, 128MB.



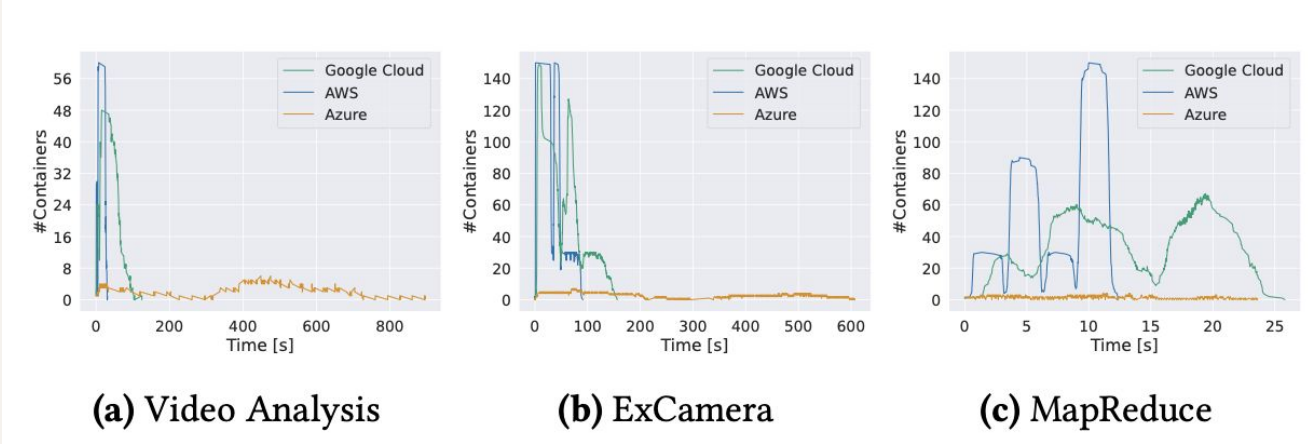


The Big Reveal

On **AWS & Google**: Runtime is dominated by the Critical Path. Overhead was tiny.

On **Azure**: Runtime was dominated by massive orchestration overhead.

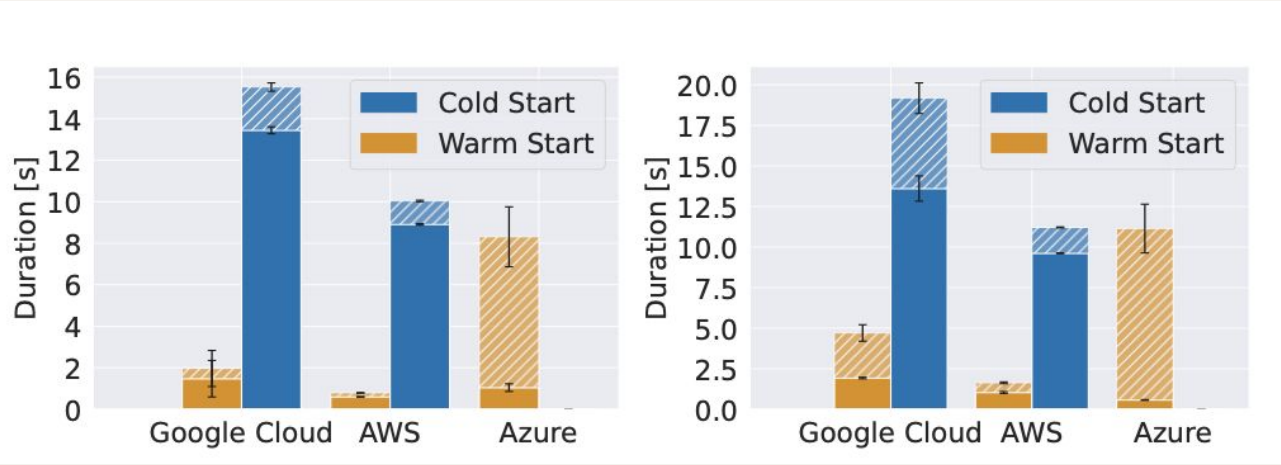
Explaining Azure's Overhead



AWS / Google Cloud: Showed **excellent scalability**, spinning up 100+ containers to meet demand.

Azure: Showed **poor scalability**, because it never scaled beyond 10 containers!

Explaining AWS & GCP Slower Compute



AWS / Google Cloud: Suffered from **constant cold starts** in 70-100% of functions.

Azure: Had **almost zero cold starts** due to its poor scalability.

What about the costs?

Finding: There is no single cheapest platform.

Total cost depends on the type of work:

- **Compute:** AWS compute is 6.7x more expensive than Google's.
- **Orchestration:** Google's model often required more steps, which added cost.
- **Storage:** For one app, storage was 28.5% of Azure's total cost vs. 2.7% for AWS .

05

The Conclusion

Why does it matter?

Conclusion

The Problem: Comparing serverless workflows was impossible due to incompatible models.

The Solution: The authors built **SeBS-Flow**, a toolkit that acts as a universal translator to enable fair comparisons.

The Finding: There is no best platform, only a **fundamental trade-off**:

- **AWS/GCP:** Great scalability, but high cold-start latency.
- **Azure:** No cold starts, but terrible scalability and orchestration overhead.

Thanks

Do you have any questions?



CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, infographics & images by **Freepik**

Please keep this slide for attribution